Parte 6

Mini-projeto: CRUD de tarefas com estado, validação, fetch e render previsível

0.1. Nesta parte

Contents

0.1. Nesta parte 2
1. Capítulo 13 — DOM e UI: formulários, estado e arquitetura de interação 3
1.1. Parte 6 — Mini-projeto: CRUD de tarefas com estado, validação, fetch e render previsível 3
1.2. 1) Requisitos 3
1.3. 2) Estrutura 3
1.4. 3) index.html (mínimo) 3
1.5. 4) app.js (state/actions/render + fetch robusto) 4
1.6. 5) O que praticar (por que isso é um CRUD profissional) 9
1.7. 6) Evoluções sugeridas 9

1. Capítulo 13 — DOM e UI: formulários, estado e arquitetura de interação

1.1. Parte 6 — Mini-projeto: CRUD de tarefas com estado, validação, fetch e render previsível

Você vai consolidar DOM + eventos + estado + validação + fetch em um mini-projeto típico: CRUD de tarefas (Create/Read/Update/Delete). O foco é arquitetura: state/actions/render, parse seguro, estados de loading/erro e prevenção de corrida (somente a última ação vale). Para praticar sem back-end próprio, usamos JSONPlaceholder (simula POST/PUT/DELETE). Em uma API real, o desenho é o mesmo.

1.2. 1) Requisitos

- Listar tarefas (GET).
- Selecionar tarefa por id e mostrar detalhes.
- Criar tarefa (POST) com validação (título mínimo).
- Editar tarefa (PUT/PATCH) com validação.
- Remover tarefa (DELETE).
- Estados: loading/erro/sucesso; botões desabilitados durante request.
- Evitar corrida: ações concorrentes não podem renderizar fora de ordem (jobId).

1.3. 2) Estrutura

```text
crud - tarefas /
  index.html
  app.js
  (styles.css opcional)
```

1.4. 3) index.html (mínimo)

```html
<! doctype html >
< html lang = "pt-BR" >
  < head >
    < meta charset = "utf-8"  />
    < meta name = "viewport"  content = "width=device-width, initial-scale=1"  />
    < title > CRUD Tarefas — Cap.  13 </ title >
    < script src = "app.js"  defer ></ script >
  </ head >
  < body >
    < h1 > CRUD de Tarefas </ h1 >

    < p id = "status" > idle </ p >
    < p id = "spinner"  style = "display:none" > ⏳ </ p >
    < pre id = "erro"  style = "color:#b00" ></ pre >
```

```

      < section >
        < h2 > Lista </ h2 >
        < button id = "reload"  type = "button" > Recarregar </ button >
        < ul id = "lista" ></ ul >
      </ section >

      < section >
        < h2 > Editor </ h2 >
        < form id = "form" >
          < label >
           Título
            < input id = "titulo"   />
            < small data - err = "titulo" ></ small >
          </ label >
          < label >
           Feita
            < input id = "done"  type = "checkbox"   />
          </ label >
          < div >
            < button id = "criar"  type = "button" > Criar </ button >
            < button id = "salvar"  type = "submit" > Salvar </ button >
            < button id = "remover"  type = "button" > Remover </ button >
          </ div >
        </ form >
        < p id = "selecionada" > Nenhuma selecionada </ p >
      </ section >
    </ body >
  </ html >
```

1.5. 4) app.js (state/actions/render + fetch robusto)

```javascript
'use strict' ;

const  API  =  'https://jsonplaceholder.typicode.com/todos' ;

function  assertEl (sel) {
  const  el  =  document. querySelector (sel);
  if  ( ! el)  throw  new  Error ( `Elemento não encontrado: ${sel}` );
  return  el;
}

 const  elStatus  =  assertEl ( '#status' );
 const  elSpinner  =  assertEl ( '#spinner' );
 const  elErro  =  assertEl ( '#erro' );
 const  btnReload  =  assertEl ( '#reload' );
 const  elLista  =  assertEl ( '#lista' );
 const  form  =  assertEl ( '#form' );
 const  elTitulo  =  assertEl ( '#titulo' );
 const  elDone  =  assertEl ( '#done' );
 const  btnCriar  =  assertEl ( '#criar' );
 const  btnSalvar  =  assertEl ( '#salvar' );
```

```
21 const btnRemover = assertEl ( '#remover' );
22 const elSelecionada = assertEl ( '#selecionada' );
23
24 let state = {
25   status : 'idle' ,
26   message : '' ,
27   fieldErrors : {},
28   items : [],
29   selectedId : null ,
30   draft : { titulo : '' , done : false },
31 };
32
33 let currentJobId = 0 ;
34 let currentController = null ;
35
36 function setState (patch) {
37   state = { ...state, ...patch };
38   render ();
39 }
40
41 function limparErros () {
42   for ( const el of document. querySelectorAll ( '[data-err]' )) el.textContent = '' ;
43 }
44
45 function renderErros (erros) {
46   limparErros ();
47   for ( const [k, v] of Object. entries (erros)) {
48     const el = document. querySelector ( `[data-err="${k}"]` );
49     if (el) el.textContent = v;
50   }
51 }
52
53 function render () {
54   elStatus.textContent = state.status;
55   elSpinner.style.display = state.status === 'loading' ? 'block' : 'none' ;
56   elErro.textContent = state.status === 'error' ? String (state.message || 'erro' ) : '' ;
57
58   const disabled = state.status === 'loading' ;
59   btnReload.disabled = disabled;
60   btnCriar.disabled = disabled;
61   btnSalvar.disabled = disabled;
62   btnRemover.disabled = disabled || state.selectedId === null ;
63
64   renderErros (state.fieldErrors);
65
66   // lista
67   elLista.innerHTML = '' ;
68   for ( const it of state.items) {
69     const li = document. createElement ( 'li' );
70     li. setAttribute ( 'data-id' , String (it.id));
71     li.textContent = `${it.completed ? '✅' : '⬜'} ${it.title}` ;
72     if (it.id === state.selectedId) li.style.fontWeight = '700' ;
73     elLista. appendChild (li);
74   }
75
76   // editor
```

```
77    elTitulo.value = state.draft.titulo;
78    elDone.checked = Boolean (state.draft.done);
79
80     const sel = state.items. find ((x) => x.id === state.selectedId) ?? null ;
81    elSelecionada.textContent = sel ? `Selecionada: #${sel.id}` : 'Nenhuma selecionada' ;
82  }
83
84  function normText (x) {
85    return String (x ?? " ). trim (). replace ( / \s +/ g, ' ' );
86  }
87
88  function validarTitulo (raw) {
89    const t = normText (raw);
90    if (t.length < 3 ) return { ok : false , msg : 'título deve ter 3+ caracteres' };
91    return { ok : true , value : t };
92  }
93
94  async function fetchWithTimeout (url, options = {}, ms = 4000 ) {
95    const controller = new AbortController ();
96    const id = setTimeout (() => controller. abort (), ms);
97    try {
98      return await fetch (url, { ...options, signal : controller.signal });
99    } finally {
100      clearTimeout (id);
101    }
102  }
103
104  async function safeReadJson (res) {
105    const text = await res. text (). catch (() => " );
106    try {
107      return { ok : true , value : JSON. parse (text) };
108    } catch {
109      return { ok : false , msg : 'JSON inválido' , raw : text };
110    }
111  }
112
113  async function apiRequest (url, options = {}) {
114    let res;
115    try {
116      res = await fetchWithTimeout (url, { headers : { Accept : 'application/
json' , ...options.headers }, ...options }, 4000 );
117    } catch (e) {
118      if (e && e.name === 'AbortError' ) return { ok : false , msg : 'timeout/cancelado' };
119      return { ok : false , msg : 'erro de rede' };
120    }
121
122    const parsed = await safeReadJson (res);
123
124    if ( ! res.ok) {
125      const msg = parsed.ok ? (parsed.value ? .error ? .message ?? res.statusText) : (parsed.raw ||
res.statusText);
126      return { ok : false , status : res.status, msg : String (msg) };
127    }
128
129    if ( ! parsed.ok) return { ok : false , status : res.status, msg : parsed.msg };
130    return { ok : true , status : res.status, data : parsed.value };
```

```
131 }
132
133 function selectById (id) {
134   const n = Number (id);
135   if ( ! Number. isFinite (n)) return ;
136   const it = state.items. find ((x) => x.id === n) ?? null ;
137   setState ({ selectedId : n, draft : { titulo : it ? it.title : " , done : it ? it.completed :
false }, fieldErrors : {} });
138 }
139
140 function updateDraftFromInputs () {
141  state = { ...state, draft : { titulo : elTitulo.value, done : elDone.checked } };
142 }
143
144 async function carregarLista () {
145  currentJobId += 1 ;
146   const jobId = currentJobId;
147   if (currentController) currentController. abort ();
148  currentController = new AbortController ();
149
150   setState ({ status : 'loading' , message : " , fieldErrors : {} });
151
152   const r = await apiRequest ( `${API}?_limit=15` , { signal : currentController.signal });
153   if (jobId !== currentJobId) return ;
154
155   if ( ! r.ok || ! Array. isArray (r.data)) {
156     return setState ({ status : 'error' , message : r.msg ?? 'falha ao carregar' });
157   }
158
159   setState ({ status : 'success' , items : r.data, message : " });
160 }
161
162 async function criar () {
163   updateDraftFromInputs ();
164   const v = validarTitulo (state.draft.titulo);
165   if ( ! v.ok) return setState ({ fieldErrors : { titulo : v.msg }, message : 'corrija o título' });
166
167   setState ({ status : 'loading' , fieldErrors : {}, message : " });
168
169   const r = await apiRequest (API, {
170    method : 'POST' ,
171    headers : { 'Content-Type' : 'application/json' },
172    body : JSON. stringify ({ title : v.value, completed : Boolean (state.draft.done), userId : 1 }),
173   });
174
175   if ( ! r.ok) return setState ({ status : 'error' , message : r.msg });
176
177   // JSONPlaceholder devolve id simulado; adiciona na lista
178   const novo = { id : r.data ? .id ?? Date. now (), title : v.value, completed : Boolean
(state.draft.done) };
179   setState ({ status : 'success' , items : [novo, ...state.items], selectedId : novo.id, message :
'criado' });
180   selectById (novo.id);
181 }
182
183 async function salvar (ev) {
```

```
184     ev. preventDefault ();
185      if  (state.selectedId  ===   null )  return ;
186
187      updateDraftFromInputs ();
188      const  v  =   validarTitulo (state.draft.titulo);
189      if  ( ! v.ok)  return   setState ({ fieldErrors :  { titulo :  v.msg }, message :   'corrija o título'  });
190
191      setState ({ status :   'loading' , fieldErrors :  {}, message :   ”  });
192
193      const  id  =  state.selectedId;
194      const  r  =   await   apiRequest ( `${API}/${id}` , {
195       method :   'PUT' ,
196       headers :  {  'Content-Type' :   'application/json'  },
197       body :  JSON. stringify ({ id, title :  v.value, completed :   Boolean (state.draft.done), userId :
1  }),
198     });
199
200      if  ( ! r.ok)  return   setState ({ status :   'error' , message :  r.msg });
201
202      const  items  =  state.items. map ((it)  =>  (it.id  ===  id  ?  { ...it, title :  v.value, completed :
Boolean (state.draft.done) }  :  it));
203      setState ({ status :   'success' , items, message :   'salvo'  });
204    }
205
206   async   function   remover () {
207      if  (state.selectedId  ===   null )  return ;
208
209      setState ({ status :   'loading' , message :   ”  });
210      const  id  =  state.selectedId;
211
212      const  r  =   await   apiRequest ( `${API}/${id}` , { method :   'DELETE'  });
213      if  ( ! r.ok)  return   setState ({ status :   'error' , message :  r.msg });
214
215      const  items  =  state.items. filter ((it)  =>  it.id  !==  id);
216      setState ({ status :   'success' , items, selectedId :   null , draft :  { titulo :   ” , done :   false  },
message :   'removido'  });
217    }
218
219   function   wire () {
220     btnReload. addEventListener ( 'click' , carregarLista);
221
222     elLista. addEventListener ( 'click' , (e)  =>  {
223       const  li  =  e.target. closest ( 'li' );
224       if  ( ! li)  return ;
225       selectById (li. getAttribute ( 'data-id' ));
226     });
227
228     elTitulo. addEventListener ( 'input' , ()  =>  {  updateDraftFromInputs ();  render (); });
229     elDone. addEventListener ( 'change' , ()  =>  {  updateDraftFromInputs ();  render (); });
230
231     btnCriar. addEventListener ( 'click' , criar);
232     form. addEventListener ( 'submit' , salvar);
233     btnRemover. addEventListener ( 'click' , remover);
234   }
235
236   wire ();
```

```
render ();
carregarLista ();
```

1.6. 5) O que praticar (por que isso é um CRUD profissional)

- Seleção por id e draft separado evitam mutação acidental e UI inconsistente.
- State controla loading/erro/sucesso e desabilita botões de forma previsível.
- Camada apiRequest centraliza status + parse seguro (não espalha fetch pelo app).
- Validação local evita requests inválidos e melhora UX.
- Delegação de eventos evita vários listeners na lista.
- jobId + abort evitam corrida e resultados fora de ordem.

1.7. 6) Evoluções sugeridas

- Adicionar filtro de busca no topo (state.q + selector derivado).
- Adicionar retry responsável em GET (429/5xx).
- Persistir draft/seleção em localStorage (cap. 10).
- Substituir JSONPlaceholder por sua API real e mapear contrato ok/data/error.
- Melhorar render (classes, estilos) sem mudar a arquitetura.

Na Parte 7, você vai fazer exercícios de refatoração e otimização: reduzir re-render desnecessário, centralizar mensagens e remover duplicação. Na Parte 8, você fecha o Capítulo 13 com resumo e checklist final.